Appendix E — Three-Run Stress Test Tables

E.1 The flag-ablation runs

Four single-session `app/agent_query.py` transcripts, run against the same 237-document corpus, each with a different slice of the pipeline's feature flags turned off, make up this stress test. Table E.1 is every real token count and iteration count from all four — including the one run that crashed.

Run	Iter. used	Quality	Prompt tok.	Compl. tok.	Total tok.
All flags True	5/6	OK	10,042	1,634	11,676
…except output-fix + draft-gen	4/6	OK	6,217	835	7,052
…except compression/dedup/retrieval-val. (attempt 1)	3/6	OK	6,407	1,160	7,567
…except compression/dedup/retrieval-val. (retry)	5/6	INSUFFICIENT	10,027	766	10,793
Post-retrieval-separation	3/6	OK	6,249	1,070	7,319

The baseline run ("Give me a complete overview of Autism Spectrum Disorder — its definition, causes, prevalence, diagnosis process, treatment options, and associated disparities") fans out into six parallel `retrieve_documents` calls in a single LLM turn. Its chunk-merge step fails its own faithfulness validator three consecutive times on one group before the pipeline gives up on merging and falls back to keeping every original chunk unmerged — a real self-correction path, not a crash:

```
[MERGE ATTEMPT 1/3] group keeper=accumulated_chunks[0], dups=[1, 2]
  [CHUNK MERGE] success -- 1241 chars, 3 source(s)
  [VALIDATE-MERGE] successfully fixed malformed LLM output.
  [VALIDATE-MERGE] verdict=UNFAITHFUL  fabricated=0  dropped=1
  [VALIDATE-MERGE]  reason:  The  MERGED  CHUNK  is  faithful  to  the  SOURCE
CHUNKS, preserving all substantive facts and not inventing any new ones.
    dropped:      The  DSM-IV  was  the  previous  edition  of  the  diagnostic
manual before the DSM-5.
  [MERGE INVALID] attempt 1 -- retrying.

  [MERGE ATTEMPT 2/3] group keeper=accumulated_chunks[0], dups=[1, 2]
  [CHUNK MERGE] retry with judge feedback (113 chars)
  [CHUNK MERGE] success -- 1431 chars, 3 source(s)
  [VALIDATE-MERGE] successfully fixed malformed LLM output.
  [VALIDATE-MERGE] verdict=UNFAITHFUL  fabricated=0  dropped=1
  [VALIDATE-MERGE]  reason:  The  merged  chunk  is  faithful  to  the  source
chunks, but it incorrectly includes the publication date of the DSM-5 as a
```

```
separate fact.
    dropped:     In 2013, the Diagnostic and Statistical Manual of Mental Disorders--5th edition (DSM-5) was published, updating the diagnostic criteria for ASD from the previous
  [MERGE INVALID] attempt 2 -- retrying.

  [MERGE ATTEMPT 3/3] group keeper=accumulated_chunks[0], dups=[1, 2]
  [CHUNK MERGE] retry with judge feedback (233 chars)
  [VALUE-VERIFY] RateLimitError -- HTTP 429: Error code: 429 - {'error': {'message': 'Rate limit reached for model `llama-3.1-8b-instant` in organization `org_01kpzxk122fm2akawhpn357gh0` service tier `on_demand` on tokens per minute (TPM): Limit 6000, Used 3905, Requested 2999. Please try again in 9.04s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
  [CHUNK MERGE] success -- 1321 chars, 3 source(s)
  [VALIDATE-MERGE] successfully fixed malformed LLM output.
  [VALIDATE-MERGE] verdict=UNFAITHFUL  fabricated=0  dropped=1
  [VALIDATE-MERGE] reason: The MERGED CHUNK is faithful to the SOURCE CHUNKS, preserving all substantive facts and not inventing any new ones.
    dropped:     The previous diagnostic criteria for ASD were in the 4th edition (DSM-IV)
  [MERGE INVALID] attempt 3 -- retrying.
  [MERGE ABANDONED] keeper=accumulated_chunks[0] -- keeping all 3 originals.

  [AGENT STATE] total_retrievals=1/5 | accumulated_chunks=5
```

Three attempts, three different judge objections (a dropped fact, an incorrectly-isolated publication date, then the same dropped fact again), and — inside attempt 3 — a real 429 rate-limit hit that the merge call survives and retries through anyway. Every attempt is judged `UNFAITHFUL` for dropping exactly one fact the merge LLM considered redundant with the DSM-5 sentence already present; the pipeline never accepts that trade-off and abandons the merge rather than installing a lossy summary.

Disabling draft creation and the answer-output fix together removes more than just the draft step: the answer-quality-check phase wired downstream of it disappears from the transcript entirely — `[DRAFT] creation disabled -- generating final answer directly.` — a real architectural coupling between two flags that look independent by name.

E.2 Where the pipeline actually breaks

The clearest failure in the whole set comes from the run with retrieval validation, dedup/merge, and every compression stage disabled at once. Its query, complete and exact, spans more than twenty subtopics in a single sentence: "Build a complete ASD knowledge graph from every document and dataset field, including definitions, symptoms, DSM criteria, screening, evaluation, genetics, epidemiology, risk factors, therapies, medications, comorbidities, severity levels, ADOS/IQ patterns, demographic disparities, socioeconomic patterns, school settings, communication strategies, sensory management, crisis handling, and source-by-source contradictions." Three of its four sub-query variants return zero chunks on the first attempt. The user marks the answer bad; on retry, with every filtering and compression stage still off, six retrievals accumulate 30 chunks — 23,572 characters — and forward all of it, unfiltered, to the draft LLM call, complete and exact:

```
  [PHASE: DRAFT]  Invoking LLM with NO tool schemas to produce answer draft.
  context: 23572 chars  |  30 chunk(s)
======================================================================
  [AGENT-DRAFT] APIStatusError -- HTTP 413: Error code: 413 - {'error': {'message': 'Request too large for model `llama-3.1-8b-instant` in organization `org_01kpzxk122fm2akawhpn357gh0` service tier `on_demand` on tokens per minute (TPM): Limit 6000, Requested 11538, please reduce your message size and try again. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
  [FAILED VARIANTS] saved 6 new failing variant(s) for this query.

======================================================================
  FINAL ANSWER
======================================================================
Draft LLM call failed: Error code: 413 - {'error': {'message': 'Request too large for model `llama-3.1-8b-instant` in organization `org_01kpzxk122fm2akawhpn357gh0` service tier `on_demand` on tokens per minute (TPM): Limit 6000, Requested 11538, please reduce your message size and try again. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
======================================================================
  Iterations used : 5 / 6
  Quality signal  : INSUFFICIENT
  Tokens (total)  : 10027 prompt + 766 completion = 10793 total
```

Common pitfall — This is why compression exists, not a hypothetical risk

This is not a new failure mode — it reproduces, six weeks later and with compression deliberately turned off, the exact class of crash Bugs.md records as BUG-F013 (“Runaway Agent Loop,” closed April 2026), whose original fix was described as “refactored compression pipeline to replace raw chunk messages with a single formatted context, preventing context window explosion.” This stress-test run turns off precisely the safeguard that fix relied on and gets the identical failure back. A companion run in the same file, the post-retrieval-separation trace with compression also off, does *not* crash — its low-specificity query only accumulates 13 chunks total, safely under the 6,000-token-per-minute ceiling. Read together, the two runs isolate the real cause: it is not “compression off” alone that breaks the pipeline, it is accumulated context size crossing the provider's limit — compression is what keeps that size bounded regardless of how broad the query is.

E.3 The 15-batch, 100-scenario workflow benchmark

A separate, larger stress test drives the LangGraph pipeline through a fixed, numbered catalog of 100 scenarios grouped into 15 batches, firing them at one long-lived `app_workflow/api.py` process with no delay between queries (ADR-070). The catalog itself — `BATCHES: dict[int, dict]` in `app/run_batch.py` — is real and present in the codebase; the standalone HTTP-driving runner script that fires it at a live server is described in detail across the project's ledgers and chat history but is not itself part of this repository snapshot, so it is cited here by description rather than quoted as source.

#	Batch	Entries
1	ASD disambiguation + thumbdown context switch	9
2	Adjustable Speed Drive deep dive, pivot back to autism	9
3	Zero-chunk-retrieval bait questions	6
4	Multi-pass, multi-source questions	6
5	Partial-knowledge-base hallucination risk	7
6	Dataset-specific analytical questions	7
7	Gender/race disparity questions	5
8	Sustained ASD/ASD-acronym domain-hint trap	7
9	Broad queries expected to load large chunk counts	4
10	Inference and applied-reasoning traps	7
11	Contradictions and knowledge-boundary gaps	6
12	Self-learning / memory pipeline tests	7
13	Multi-turn thumbdown refinement	6
14	Quality-checker and grounding stress tests	4
15	Final mixed and edge-case scenarios	10

The catalog deliberately keeps the same domain-disambiguation trap running throughout — Autism Spectrum Disorder and Adjustable Speed Drive share the acronym “ASD” only informally, but the corpus mixes both topics on purpose to test whether the pipeline confuses them. Batch 1, in full and exactly as written in `app/run_batch.py`, is the clearest illustration — a scripted thumbdown mid-batch forces a topic-switch context-recovery test, followed immediately by the same question asked twice more from opposite domains:

```
1: {
    "name": "asd_disambiguation_thumbdown",
    "description": "ASD disambiguation + thumbdown context switch (Q1-Q9)",
    "questions": [
        "What is ASD and what are its main characteristics?",
        "How is ASD diagnosed in children under 3 years old?",
        "What comorbid conditions are commonly found alongside ASD?",
        "Can ASD be caused by vaccines?",
        "What therapies are recommended for someone with ASD?",
        "Does ASD affect males and females equally?",
        {
            "type": "bad",
            "feedback": (
                "I was actually asking about Adjustable Speed Drives, not
autism. "
                "All your answers were about the wrong topic."
            ),
            "_note": "Q7 -- context switch thumbdown",
        },
        "What is ASD and what are its main characteristics?",
        "How does an ASD control the speed of an electric motor?",
    ],
},
```

Batch 12's self-learning probe follows the same real, scripted pattern of mixing plain questions with typed commands the runner dispatches to `/stats` and `/learn` rather than `/query` — ask a question, check `/stats`, force `/learn`, check `/stats` again, then re-ask the same question to see whether distillation changed the answer.

E.4 Per-stage timing — the millisecond-to-13-minute long tail

`timing_tracker.py` records nine pipeline stages as flat, ever-growing lists of durations in seconds, flushed to JSON after every single measurement. Two real `timing_tracker` artifacts survive from actual benchmark runs. The smaller of the two — `Runs/langgraph_api_20260714_174519.debug.json`, the file Research.md's own benchmark write-up cites by name — is reproduced here in full, every value, exactly as recorded:


```
{
"Sub-Query Generation": [
  3.2271, 2.4146, 2.5754, 2.8385, 2.509, 2.9879, 3.1046, 3.0884, 3.0316,
1.802,
  2.7134, 1.6715, 4.6217, 5.2022, 2.1223, 2.1078
],
"Total DB Retrieval Time": [
  0.0974, 0.1055, 0.1102, 0.203, 0.1975, 0.2242, 0.2109, 0.2378, 0.2366,
0.0859,
  0.1106, 0.1117, 0.1172, 0.1284, 0.1344, 0.1889, 0.1914, 0.1964, 0.1358,
0.143,
  0.1644, 0.1345, 0.1505, 0.1611, 0.0919, 0.1006, 0.3136, 0.3399, 0.3455,
0.0689,
  0.0761, 0.1167, 0.1375, 0.1325, 0.0639, 0.0676, 0.1912, 0.6973, 0.1713,
0.2297,
  0.0735, 0.0872
],
"Total DB Retrieval Validation Time": [
  40.5706, 54.4457, 71.1675, 96.0448, 48.6547, 67.3781, 40.9239, 56.4594,
73.3776, 101.7332,
  0.0029, 0.0037, 0.0019, 0.0017, 0.0027, 0.0042, 10.0599, 15.8705,
15.2308, 23.7112,
  20.1209, 34.1617, 44.5868, 60.546, 0.0019, 0.0025, 0.001, 0.0017, 0.0032,
0.0034,
  32.2818, 43.1717
],
"Total Merge Time for Retrieved Chunks": [
  21.6318, 310.2123, 133.968, 141.4088, 17.462, 21.6828, 20.9168, 27.4066,
49.254, 63.2943,
  0.0037, 0.005, 0.0011, 0.002, 0.004, 0.0042, 0.0041, 5.7812, 0.0033,
0.0027,
  0.0047, 0.0039, 22.2095, 35.802, 0.0021, 0.0027, 0.0016, 0.5967, 0.0028,
0.0027,
  86.6289, 114.0274
],
"Total Validation Time for Merged Chunks": [
  41.4473, 95.358, 358.9809, 373.8757, 32.7765, 34.2934, 43.4979, 51.3049,
53.9433, 76.5919,
  0.0019, 0.003, 0.0018, 0.0061, 0.0017, 0.0021, 0.004, 11.3986, 0.0016,
```

```
0.0018,
  0.0005, 0.0006, 26.0999, 39.3154, 0.0008, 0.0015, 0.0014, 0.002, 0.0005,
0.0009,
  109.805, 120.8463
],
"Compression": [
  0.02, 35.5456, 0.0064, 8.7522, 757.5297, 783.6073, 9.9395, 41.3893,
46.3741, 19.9058,
  37.809, 69.6234, 34.3745, 38.9163, 32.3293, 71.2508, 9.2295, 25.1899,
43.3129, 24.7849,
  0.003, 0.0118, 0.0091, 0.0035, 26.3728, 29.719, 10.3293, 11.7452,
19.2821, 16.3099,
  0.0124, 6.7603, 0.004, 4.2184, 26.102, 31.7901, 9.7184, 11.1746, 24.1476,
22.7953,
  0.009, 11.568, 0.0054, 4.3159, 39.9758, 41.7874, 14.9637, 16.3033,
23.6482, 24.6538,
  0.0033, 0.0066, 0.0034, 0.0041, 0.0048, 0.0072, 0.0032, 0.0037, 0.003,
0.0022,
  0.0688, 0.1122, 0.0023, 0.0018, 0.0643, 0.0651, 0.0037, 0.0046, 0.0031,
0.0013,
  0.0025, 0.0051, 0.0134, 0.0081, 0.0134, 0.0121, 0.0036, 0.0037, 0.0025,
0.0021,
  0.0024, 0.0111, 0.004, 0.0048, 0.0128, 3.8318, 0.0317, 2.995, 0.0027,
0.002,
  0.0035, 0.0056, 0.0027, 0.0044, 0.0051, 0.005, 0.0052, 0.0057, 0.0039,
0.0025,
  0.0025, 0.0068, 0.0028, 0.0023, 0.0185, 0.0187, 0.0036, 0.0037, 0.0019,
0.0021,
  23.438, 0.0071, 7.5454, 0.0127, 24.1588, 0.0081, 22.7747, 7.7129, 7.8507,
0.0093,
  0.0117, 0.0019, 0.0017, 0.0031, 0.0058, 0.0025, 0.003, 0.0019, 0.0017,
0.0014,
  0.4097, 0.0017, 0.0016, 0.0044, 0.4252, 0.0021, 0.003, 0.0016, 0.0016,
0.0022,
  0.0017, 5.1295, 5.1307, 0.0048, 0.0063, 0.0031, 0.0034, 0.0021, 0.002,
0.0118,
  723.2227, 0.0275, 0.017, 49.3608, 69.6676, 14.8878, 270.5333, 30.2541,
0.0015
],
"Draft Generation": [
  5.7296, 4.3679, 3.8186, 3.0678, 4.5962, 1.172, 1.3219, 24.3807
],
"CAQ": [
  9.2025, 4.0379, 3.7401, 7.9639, 3.9388, 4.2666, 3.8215, 34.6934
],
"Final Generation": [
```

```
  3.4612, 3.1905, 3.4757, 1.282
]
}
```


A second, larger companion artifact — `Runs/langgraph_api_20260714_114539.debug.json` — records a longer run (Compression alone holds 817 individual measurements, versus 159 above) and is summarized rather than reproduced here in full for that reason; Table E.4 below is computed from both files together, using the identical n/min/max/sum arithmetic applied to the complete array quoted above.

Stage	n	min (s)	max (s)	sum (s)
Sub-Query Generation	82	0.12	5.38	229.0
Total DB Retrieval Time	221	0.04	1.10	25.1
Total DB Retrieval Validation Time	164	0.001	199.74	6,890.9
Total Merge Time for Retrieved Chunks	164	0.001	311.07	3,418.8
Total Validation Time for Merged Chunks	164	0.0003	159.10	3,801.3
Compression	817	0.0004	830.46	10,713.7
Draft Generation	54	0.70	8.70	165.6
CAQ	54	3.12	10.44	264.7
Final Generation	36	0.79	8.23	101.2

That 830.46-second maximum in the Compression row is thirteen minutes and forty-two seconds for a single compression call — real, present in the raw JSON array, and the single largest measurement in either file. A smaller companion run recorded a comparable extreme (783.61 s, roughly 13.06 minutes), which is the figure Research.md's own benchmark write-up describes as “individual aggregate compression measurements exceed 12 minutes” — this appendix's number is the exact value behind that rounded prose. Every other stage's *minimum* sits under a second, and most medians do too; it is specifically the compression stage's worst case, not its typical case, that stretches into minutes — the long tail the flag-ablation guards in §E.2 exist to keep bounded.

> **Common pitfall — These arrays carry no scenario tag**
>
> `timing_tracker`'s JSON output is a flat, chronologically-ordered list per stage — no request ID, no scenario number, no timestamp attached to any individual measurement. The n/min/max/sum figures above are real and directly computed from the raw files, but tracing which specific one of the 100 scenarios produced the 830-second outlier is not possible from this artifact alone — that would require cross-referencing timestamps against the much larger companion debug logs those same runs produced.